

Actividad del Tema 6

CRUD de Productos en Laravel

Javier Beteta

2025

Práctica: Desarrollo de un CRUD de Productos en Laravel

Objetivo de la actividad



Figure 1: Actividad en el aula virtual

El objetivo de esta práctica es aprender a **aplicar formularios, rutas, controladores y vistas** en Laravel para desarrollar un **CRUD completo para el recurso *Product***.

Con esta actividad, aprenderás a trabajar con el patrón **MVC (Modelo-Vista-Controlador)**, a gestionar datos con formularios y a usar las herramientas de Eloquent para crear, leer, actualizar y eliminar productos en la base de datos. Además, practicarás la implementación de vistas con **Blade** y el uso de rutas dinámicas.

Al finalizar, deberás ser capaz de:

- Crear y ejecutar migraciones para definir la estructura de una base de datos.
- Implementar un controlador con todos los métodos CRUD.
- Crear vistas Blade para presentar datos y manejar formularios.
- Gestionar datos de entrada y salida mediante formularios, con las directivas `@csrf` y `@method`.

Instrucciones

Sigue los pasos a continuación. Verifica que todo funciona correctamente antes de pasar al siguiente paso. Si alguna parte del proceso no está clara, consulta la documentación oficial o busca ejemplos dentro del proyecto que has trabajado anteriormente.

1. Preparar el entorno

1. **Accede a tu proyecto Laravel.** Si no tienes un proyecto en marcha, puedes usar el que creaste en el Tema 4 o crear uno nuevo con `laravel new nombre_del_proyecto`.
2. **Inicia el servidor.** Asegúrate de que tu servidor esté funcionando, inicia `docker compose up -d`.
3. **Crea una nueva migración y modelo para el recurso Product.** Utiliza el comando de Artisan para crear un modelo `Product` junto con su migración. Esto generará un archivo para crear la tabla de productos en la base de datos.

2. Crear la tabla y el modelo

1. **Edita la migración de productos.** Abre el archivo de migración generado y define los siguientes campos para la tabla `products`. Estos campos incluyen el nombre del producto, una descripción, precio, stock y las fechas de creación/actualización automáticas.

La estructura de la tabla deberá ser:

Campo	Tipo de dato	Restricciones / Descripción
<code>id</code>	<code>bigIncrements</code>	Clave primaria, autoincremental
<code>name</code>	<code>string(255)</code>	Nombre del producto, no nulo
<code>description</code>	<code>text</code>	Descripción del producto
<code>price</code>	<code>decimal(8,2)</code>	Precio del producto, no nulo
<code>stock</code>	<code>integer</code>	Cantidad en stock, no nulo
<code>created_at</code>	<code>timestamp</code>	Fecha de creación (automático)
<code>updated_at</code>	<code>timestamp</code>	Fecha de actualización (automático)

2. **Ejecuta la migración.** Una vez hayas definido los campos, ejecuta la migración para crear la tabla en la base de datos.

Usa el comando:

```
php artisan migrate
```

Verifica que la tabla se haya creado correctamente en la base de datos.

3. **Configura el modelo `Product`.** En el archivo del modelo `Product`, debes definir las propiedades que permiten la asignación masiva de los campos. Esto se hace con la propiedad `$fillable`:

```
protected $fillable = ['name', 'description', 'price', 'stock'];
```

3. Crear el controlador *resource*

1. **Genera el controlador de recursos.** Utiliza Artisan para crear un controlador de recursos. Esto generará automáticamente los métodos necesarios para manejar las operaciones CRUD. Los métodos a implementar serán: `index`, `create`, `store`, `show`, `edit`, `update` y `destroy`.

El comando es:

```
php artisan make:controller ProductController --resource
```

2. **Configura el controlador.** En el archivo generado `ProductController.php`, implementa la lógica necesaria para gestionar los productos. Recuerda que Laravel ya genera los métodos básicos (`index`, `create`, `store`, etc.), pero tendrás que completar la lógica específica de cada uno.

- **`index`:** Muestra todos los productos.
- **`create`:** Muestra el formulario para crear un nuevo producto.
- **`store`:** Guarda el nuevo producto en la base de datos.
- **`show`:** Muestra los detalles de un producto específico.
- **`edit`:** Muestra el formulario para editar un producto.
- **`update`:** Actualiza un producto existente en la base de datos.
- **`destroy`:** Elimina un producto de la base de datos.

4. Definir las rutas

1. **Declara las rutas necesarias para el CRUD.** En `routes/web.php`, añade la ruta de tipo *resource* para el controlador `ProductController`. Laravel generará automáticamente todas las rutas necesarias.

Agrega esta línea:

```
Route::resource('product', ProductController::class);
```

Esto creará las rutas necesarias para manejar las operaciones CRUD para el recurso **Product**.

2. **Verifica que las rutas se han registrado correctamente.** Usa el comando `php artisan route:list --path=product` para asegurarte de que las rutas están definidas correctamente.

5. Crear las vistas

1. **Crea la carpeta para las vistas.** En `resources/views`, crea una nueva carpeta llamada `products`. En esta carpeta crearás las vistas para las operaciones CRUD.
2. **Vistas necesarias:** Crea las siguientes vistas dentro de la carpeta `products/`:
 - **index.blade.php:** Muestra todos los productos en una tabla. Esta vista debe listar todos los productos y permitir enlaces para crear, editar y eliminar productos.
 - **create.blade.php:** Formulario para crear un nuevo producto.
 - **edit.blade.php:** Formulario para editar un producto existente.
 - **show.blade.php:** Muestra los detalles de un producto.

6. Probar el CRUD completo

1. Accede a `/product` para ver el listado de productos.
2. Crea un nuevo producto y verifica que aparece en el listado.
3. Edita un producto y confirma que se actualiza correctamente.
4. Muestra los detalles de un producto desde el enlace.
5. Elimina un producto y asegúrate de que desaparezca del listado.

7. CSS (opcional)

Puedes mejorar la apariencia de tu aplicación utilizando CSS. Puedes crear un archivo CSS para la vista `index.blade.php` y otro para los formularios `create.blade.php`, `edit.blade.php` y `show.blade.php`. Asegúrate de enlazar estos archivos CSS en las vistas correspondientes.

Entrega

Sube a la plataforma de Aules:

- **Código del controlador `ProductController`.**
- **Vistas** completas para cada operación (index, create, edit, show).
- **Capturas de pantalla** mostrando cada operación CRUD funcionando:
 - Listado de productos.
 - Formulario de creación de producto.
 - Formulario de edición de producto.
 - Vista de detalles del producto.
- **Evidencia de que las operaciones CRUD funcionan correctamente.**

Rúbrica de evaluación

Criterio	Descripción	Peso
Rutas definidas	Las rutas CRUD están definidas correctamente en <code>routes/web.php</code> .	20 %
Controlador implementado	El controlador <code>ProductController</code> tiene los métodos CRUD implementados correctamente.	25 %
Vistas Blade	Las vistas están correctamente estructuradas para mostrar y manejar los productos.	25 %
Prueba del CRUD	El CRUD funciona correctamente: crea, edita, muestra y elimina productos sin errores.	20 %
Claridad en la entrega	El código está organizado y la entrega incluye capturas de pantalla claras de cada operación CRUD.	10 %

Total: 100 %